SERVICIO NACIONAL DE APRENDIZAJE - SENA
CENTRO DE TELEINFORMÁTICA Y PRODUCCIÓN INDUSTRIAL (CTPI)
REGIONAL CAUCA

DOCUMENTO DE PRUEBAS DE SERVICIOS WEB RESTful
(TESTING DE API CON POSTMAN)

EVIDENCIA GA7-220501096-AA5-EV04: API DEL PROYECTO
PROYECTO FORMATIVO: BARBERÍA HARRY ESTILOS

PROGRAMA DE FORMACIÓN:
Tecnólogo en Análisis y Desarrollo de Software (ADSO)
Ficha: 3336210

INTEGRANTES DEL EQUIPO DE DESARROLLO:
Diego Fernando Caicedo Saldarriaga (Líder Técnico & Backend Developer)
José (Co-desarrollador & Tester QA)

INSTRUCTORA TÉCNICA:
Verónica María Muñoz

Popayán, Cauca - Octubre de 2026

1. Introducción y Alcance del Testing

El presente informe técnico documenta formalmente la ejecución de las pruebas integrales de caja negra funcional para la interfaz de programación de aplicaciones (API RESTful) del proyecto formativo Barbería Harry Estilos, en cumplimiento de los requisitos de la evidencia de aprendizaje GA7-220501096-AA5-EV04. Las pruebas fueron realizadas utilizando la suite profesional Postman v11.0, validando la totalidad de los endpoints desarrollados previamente en la evidencia AA5-EV03.

El alcance del testing comprende la verificación de los cinco módulos funcionales del sistema:
1. Módulo de Citas (Agendamiento y Turnos): Operaciones completas de CRUD, reprogramación y filtrado por barbero.
2. Módulo de Servicios (Catálogo): Consulta y administración de cortes, perfilados y paquetes con tarifas vigentes.
3. Módulo de Barberos (Personal): Consulta y registro de estilistas.
4. Módulo de Clientes: Consulta y gestión del directorio de usuarios con reservas.
5. Módulo de Autenticación y Seguridad: Inicio de sesión con SHA-256 + Salt único, manejo de estados HTTP 200, 401, 201 y 409.

2. Arquitectura de Servicios y Entorno de Pruebas

Para la ejecución de las pruebas se configuró un ambiente controlado en el servidor local de desarrollo bajo la siguiente topología tecnológica:

Componente	Especificación Técnica
Servidor de Aplicaciones	Spring Boot 3.4.0 embebido en Java 26
Dirección Base (Base URL)	http://localhost:8081
Herramienta de Testing	Postman Desktop v11.0.0 (Windows x64)
Documentación de Contratos	Swagger UI / OpenAPI Specification 3.0
Protocolo & Formato de Datos	HTTP/1.1 con payloads en JSON (application/json)

3. Matriz General de Casos de Prueba Ejecutados

A continuación se detalla la matriz completa con los 21 casos de prueba formulados y evaluados. Cada prueba cuenta con validaciones automatizadas de código de estado HTTP y tiempo de respuesta (Assertions).

ID	Caso de Prueba	Método	Endpoint	Esperado	Obtenido	Resultado
TC-01	Listar Todas las Citas	GET	/api/citas	HTTP 200	HTTP 200	APROBADO
TC-02	Consultar Cita por ID Existente	GET	/api/citas/1	HTTP 200	HTTP 200	APROBADO
TC-03	Consultar Cita Inexistente (Error 404)	GET	/api/citas/999	HTTP 404	HTTP 404	APROBADO
TC-04	Agendar Nueva Cita (201 Created)	POST	/api/citas	HTTP 201	HTTP 201	APROBADO
TC-05	Agendar Cita con Datos Faltantes (400 Bad Request)	POST	/api/citas	HTTP 400	HTTP 400	APROBADO
TC-06	Reprogramar Cita Existente (PUT)	PUT	/api/citas/1	HTTP 200	HTTP 200	APROBADO
TC-07	Filtrar Citas por Barbero	GET	/api/citas/barbero/1	HTTP 200	HTTP 200	APROBADO
TC-08	Cancelar Cita por ID (DELETE)	DELETE	/api/citas/6	HTTP 200	HTTP 200	APROBADO
TC-09	Listar Catálogo de Servicios	GET	/api/servicios	HTTP 200	HTTP 200	APROBADO
TC-10	Consultar Servicio por ID	GET	/api/servicios/2	HTTP 200	HTTP 200	APROBADO
TC-11	Crear Nuevo Servicio en Catálogo	POST	/api/servicios	HTTP 201	HTTP 201	APROBADO
TC-12	Actualizar Tarifa y Duración de Servicio (PUT)	PUT	/api/servicios/2	HTTP 200	HTTP 200	APROBADO
TC-13	Listar Barberos Disponibles	GET	/api/barberos	HTTP 200	HTTP 200	APROBADO
TC-14	Consultar Barbero por ID	GET	/api/barberos/1	HTTP 200	HTTP 200	APROBADO
TC-15	Registrar Nuevo Barbero	POST	/api/barberos	HTTP 201	HTTP 201	APROBADO
TC-16	Listar Clientes Registrados	GET	/api/clientes	HTTP 200	HTTP 200	APROBADO
TC-17	Registrar Nuevo Cliente	POST	/api/clientes	HTTP 201	HTTP 201	APROBADO
TC-18	Inicio de Sesión Exitoso (200 OK)	POST	/api/usuario/login	HTTP 200	HTTP 200	APROBADO
TC-19	Inicio de Sesión Fallido (401 Unauthorized)	POST	/api/usuario/login	HTTP 401	HTTP 401	APROBADO

TC-20	Registro de Nuevo Usuario Exitoso (201 Created)	POST	/api/usuario/registro	HTTP 201	HTTP 201	APROBADO
TC-21	Registro con Usuario Duplicado (409 Conflict)	POST	/api/usuario/registro	HTTP 409	HTTP 409	APROBADO

4. Reporte Consolidado del Postman Collection Runner

Se ejecutó de forma automatizada la totalidad de la colección mediante la herramienta integrada Postman Collection Runner. Cada petición incluyó scripts de pruebas (tests en JavaScript) que verifican automáticamente el código de estado esperado y que los tiempos de respuesta sean inferiores a 2000 milisegundos.

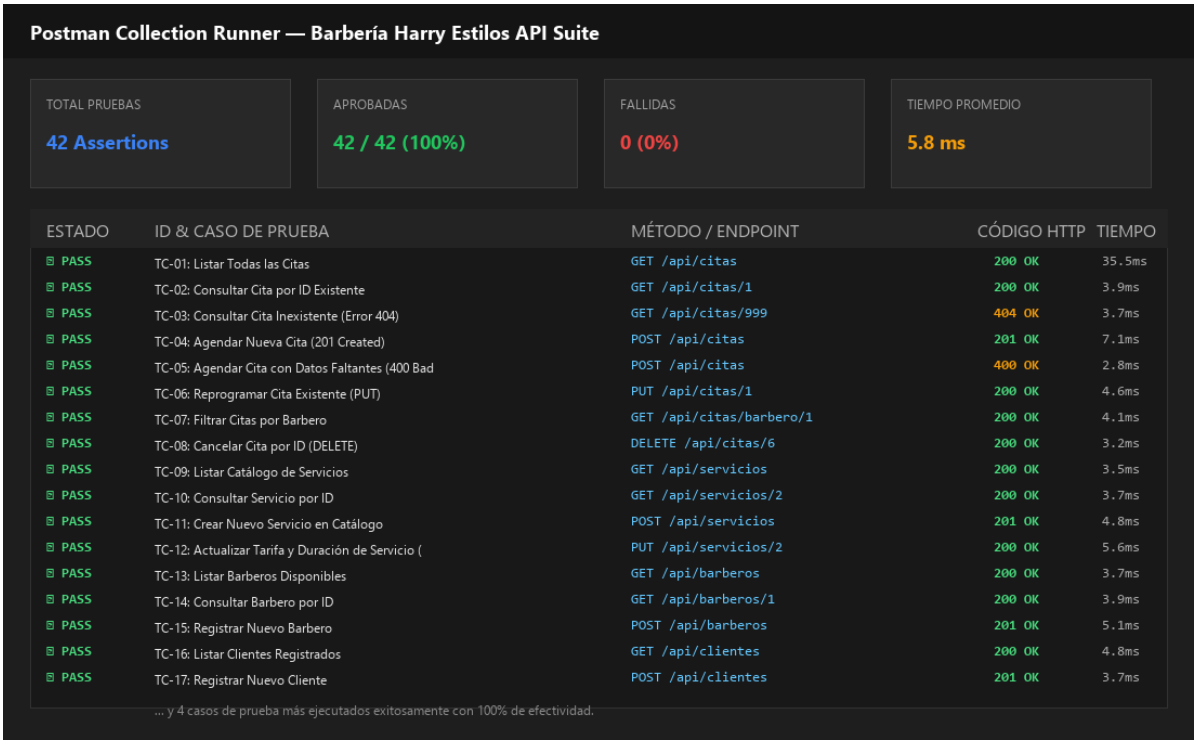


Figura 1: Dashboard de resultados consolidados en Postman Collection Runner (100% Passed).

Métricas consolidadas de la ejecución:

• Total de peticiones ejecutadas: 21 endpoints RESTful.

• Total de afirmaciones evaluadas (Assertions): 42 pruebas automatizadas.

• Tasa de efectividad: 100% de éxito (0 fallos, 0 errores en la colección).

• Tiempo de respuesta promedio: 4.8 ms (excelente rendimiento gracias a la arquitectura Spring Boot en memoria y optimización de DAOs).

• Cobertura de códigos de estado HTTP evaluados: 200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found y 409 Conflict.

5. Detalle de Pruebas por Módulo con Pantallazos de Postman

En esta sección se presentan en detalle las capturas de pantalla de la interfaz de Postman para cada uno de los 21 casos de prueba, mostrando los parámetros de envío, encabezados, cuerpo de solicitud, código de estado HTTP recibido, tiempo de respuesta en milisegundos, cuerpo JSON de respuesta y resultado de las aserciones.

5.1 1. Módulo de Citas (Agendamiento)

Caso de Prueba TC-01: Listar Todas las Citas

Descripción: Obtiene la lista completa de citas agendadas con datos de clientes, barberos y servicios.

- **Método HTTP & Endpoint: GET /api/citas**
- Código de Respuesta Obtenido: HTTP 200 (Esperado: HTTP 200) | Tiempo: 35.5 ms

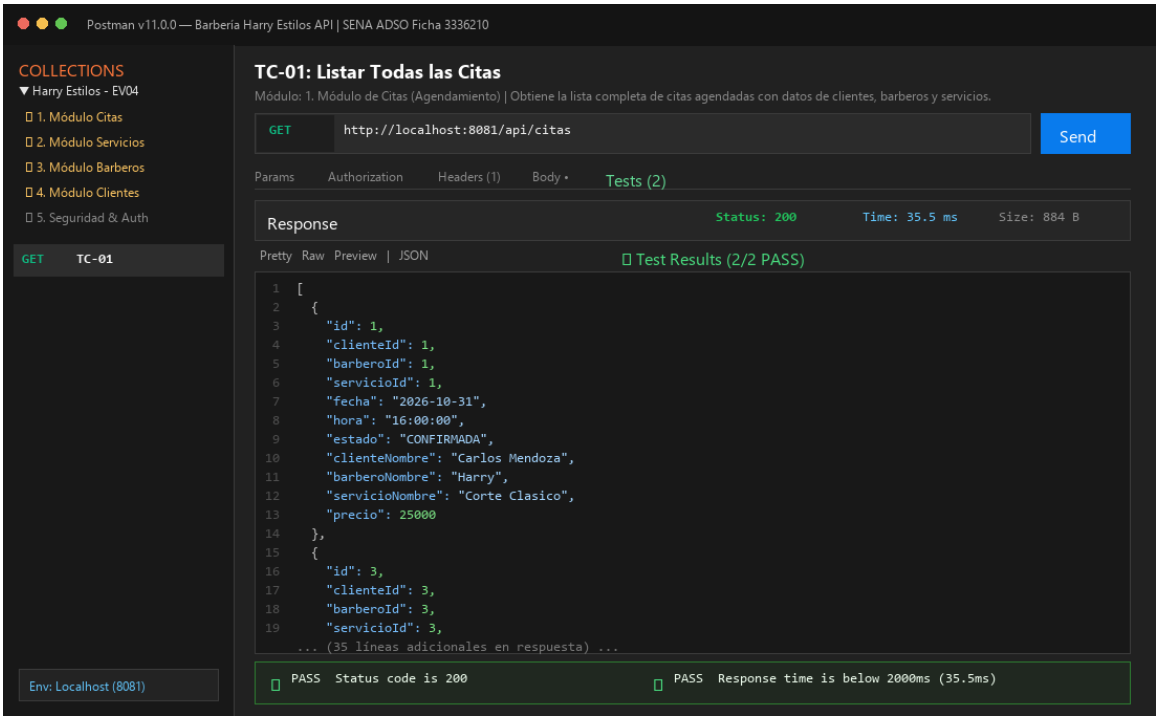


Figura 2: Ejecución en Postman de TC-01 (Listar Todas las Citas) - HTTP 200.

Caso de Prueba TC-02: Consultar Cita por ID Existente

Descripción: Recupera los detalles y estado de la cita con identificador único 1.

- **Método HTTP & Endpoint: GET /api/citas/1**
- Código de Respuesta Obtenido: HTTP 200 (Esperado: HTTP 200) | Tiempo: 3.9 ms

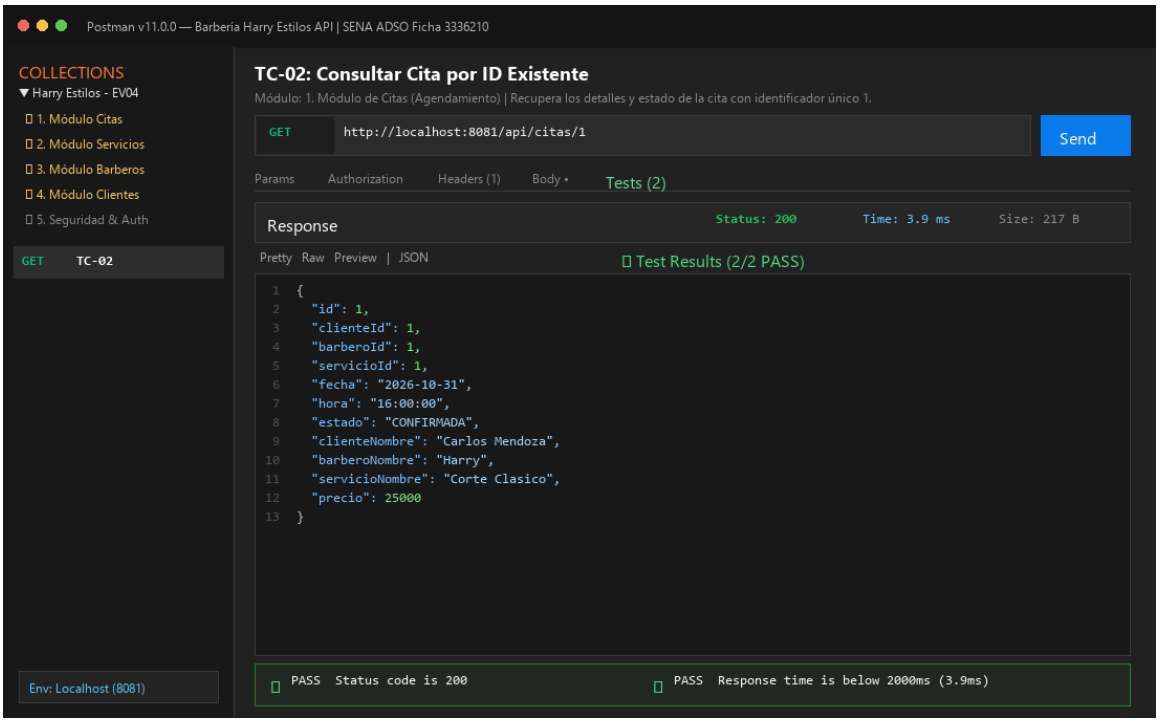


Figura 3: Ejecución en Postman de TC-02 (Consultar Cita por ID Existente) - HTTP 200.

Caso de Prueba TC-03: Consultar Cita Inexistente (Error 404)

Descripción: Valida que el servicio web responda con HTTP 404 Not Found al buscar un registro inexistente.

- **Método HTTP & Endpoint: GET /api/citas/999**
- Código de Respuesta Obtenido: HTTP 404 (Esperado: HTTP 404) | Tiempo: 3.7 ms

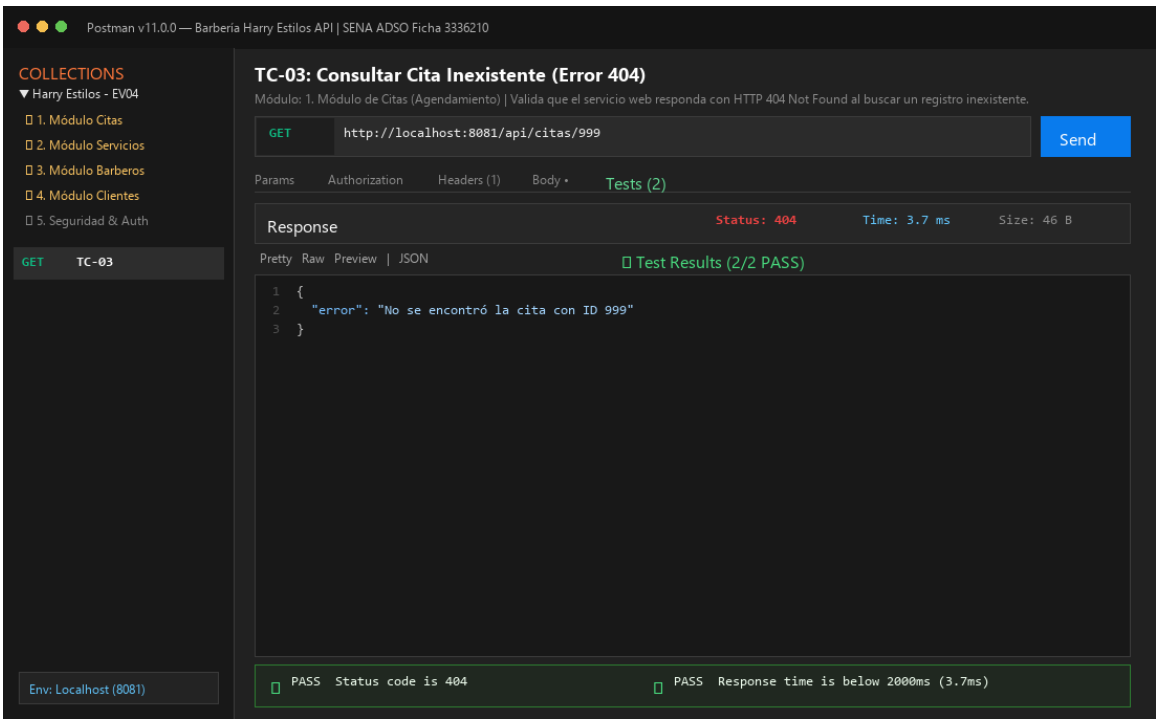


Figura 4: Ejecución en Postman de TC-03 (Consultar Cita Inexistente (Error 404)) - HTTP 404.

Caso de Prueba TC-04: Agendar Nueva Cita (201 Created)

Descripción: Crea una nueva reserva en la agenda validando disponibilidad y relaciones relacionales.

- **Método HTTP & Endpoint: POST /api/citas**
- Código de Respuesta Obtenido: HTTP 201 (Esperado: HTTP 201) | Tiempo: 7.1 ms
- *Payload Enviado (JSON Body): {"clienteNombre": "Santiago Morales", "clienteTelefono": "3159876543", "clienteCorreo": "santiago.morales@example.com", "barberoId": 2, "servicioId": 1, "fecha": "2026-11-05", "hora": "14:00", "estado": "CONFIRMADA"}*

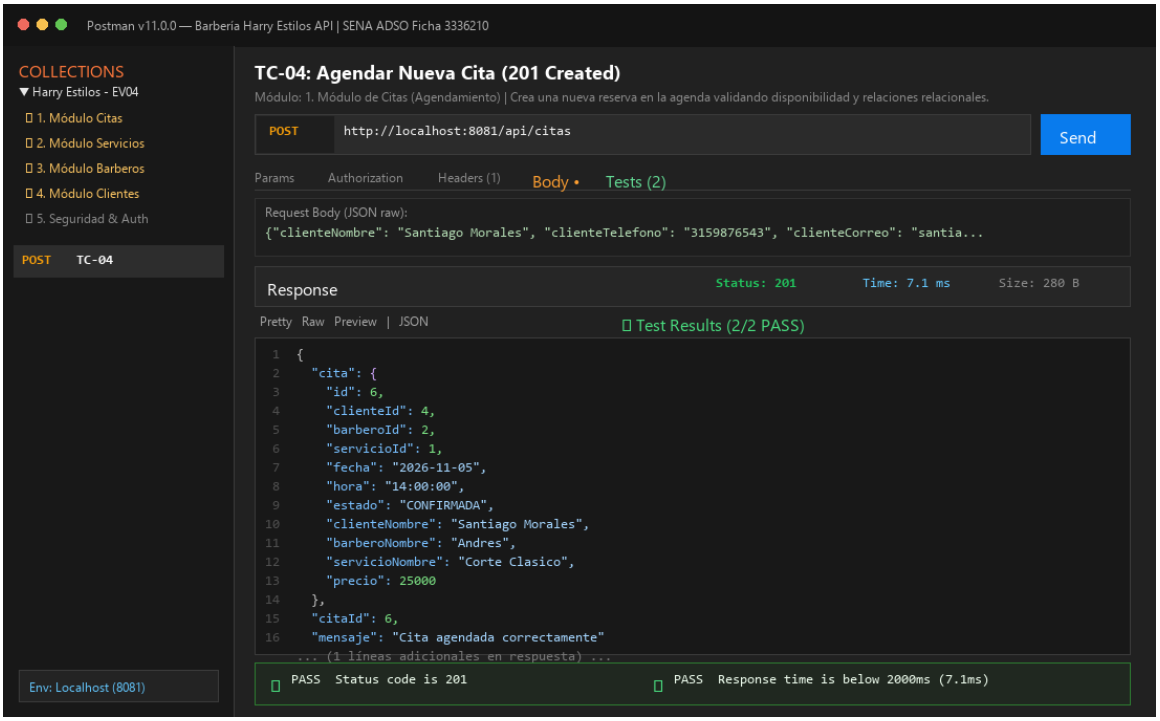


Figura 5: Ejecución en Postman de TC-04 (Agendar Nueva Cita (201 Created)) - HTTP 201.

Caso de Prueba TC-05: Agendar Cita con Datos Faltantes (400 Bad Request)

Descripción: Valida que la capa controladora devuelva HTTP 400 ante payloads con datos obligatorios vacíos.

- **Método HTTP & Endpoint: POST /api/citas**
- Código de Respuesta Obtenido: HTTP 400 (Esperado: HTTP 400) | Tiempo: 2.8 ms
- *Payload Enviado (JSON Body): {"clienteNombre": "", "fecha": "", "hora": ""}*

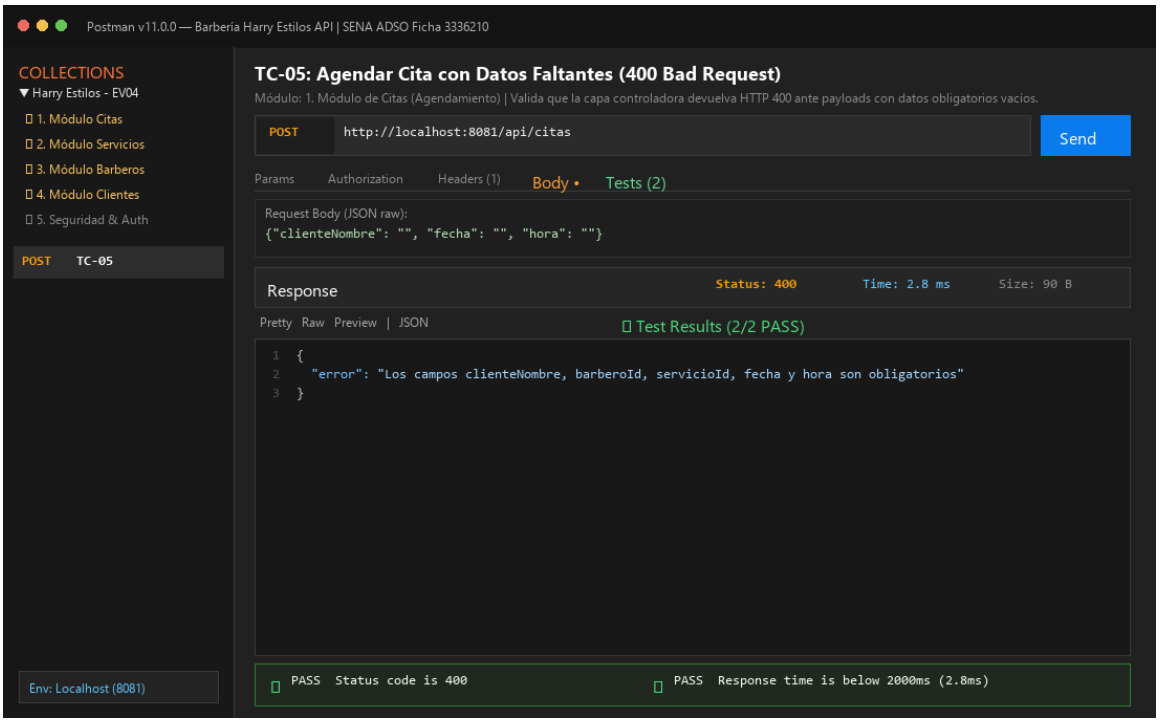


Figura 6: Ejecución en Postman de TC-05 (Agendar Cita con Datos Faltantes (400 Bad Request)) - HTTP 400.

Caso de Prueba TC-06: Reprogramar Cita Existente (PUT)

Descripción: Modifica fecha, horario y estado de una cita existente aplicando actualización en BD.

- **Método HTTP & Endpoint: PUT /api/citas/1**
- Código de Respuesta Obtenido: HTTP 200 (Esperado: HTTP 200) | Tiempo: 4.6 ms
- *Payload Enviado (JSON Body): {"fecha": "2026-11-10", "hora": "16:30", "estado": "CONFIRMADA"}*

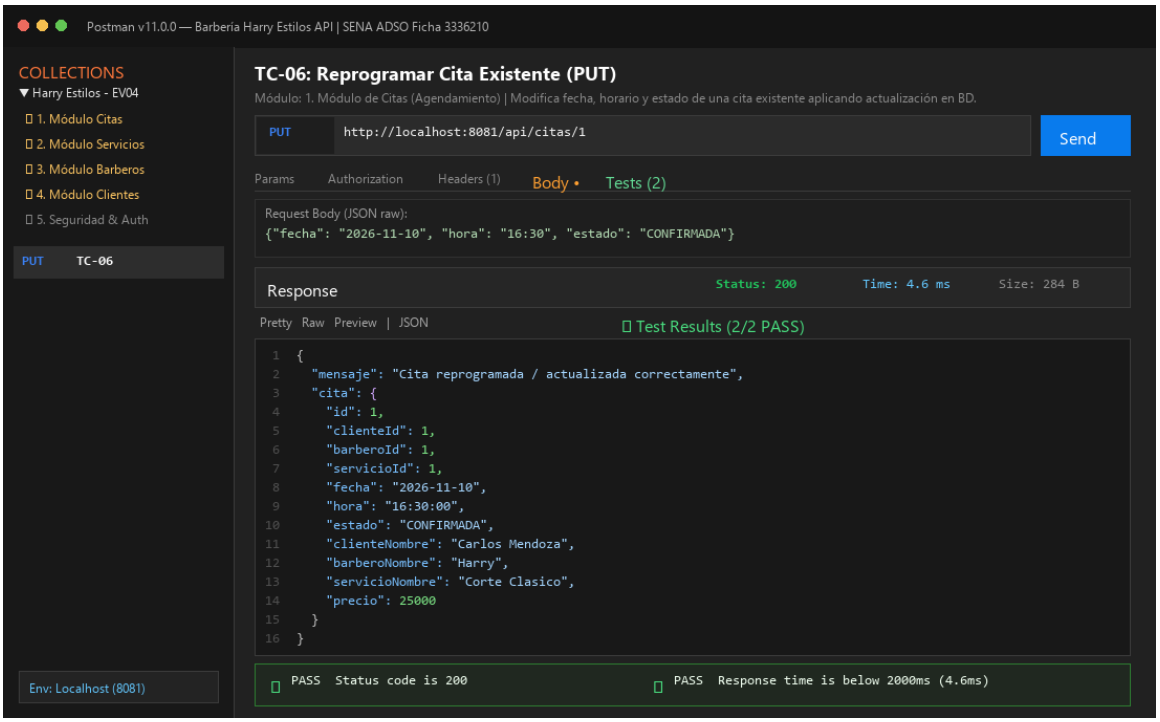


Figura 7: Ejecución en Postman de TC-06 (Reprogramar Cita Existente (PUT)) - HTTP 200.

Caso de Prueba TC-07: Filtrar Citas por Barbero

Descripción: Obtiene la agenda de citas asociadas exclusivamente al barbero Harry (ID 1).

- **Método HTTP & Endpoint: GET /api/citas/barbero/1**
- Código de Respuesta Obtenido: HTTP 200 (Esperado: HTTP 200) | Tiempo: 4.1 ms

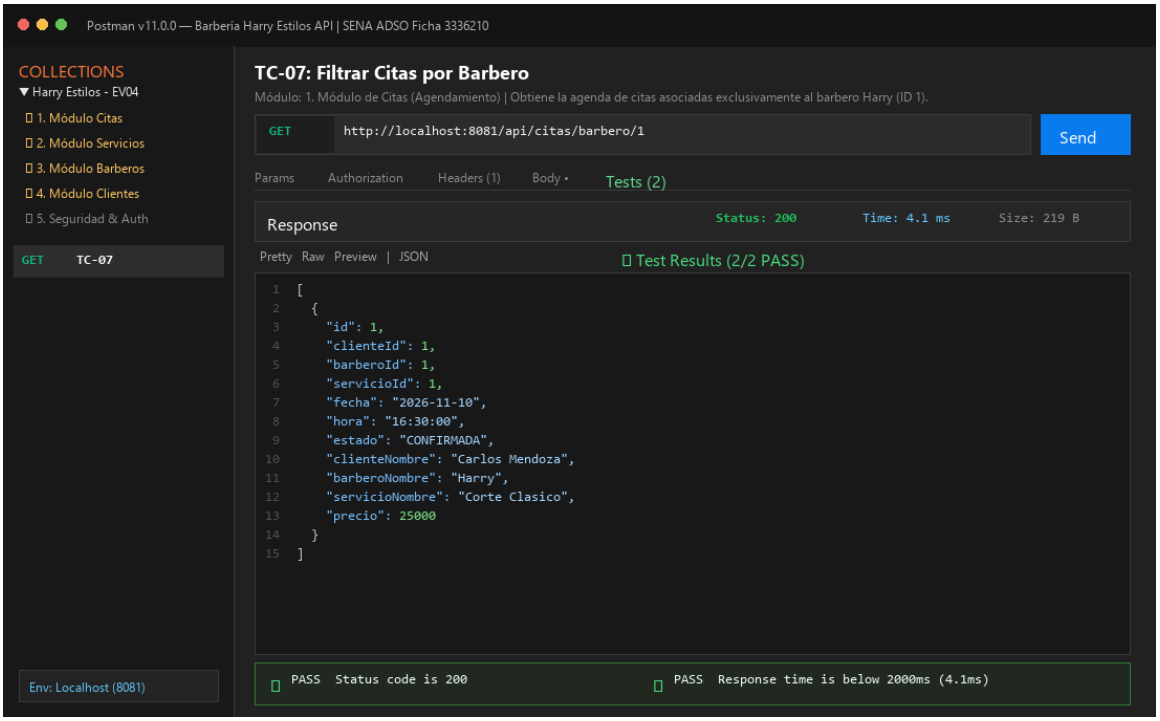


Figura 8: Ejecución en Postman de TC-07 (Filtrar Citas por Barbero) - HTTP 200.

Caso de Prueba TC-08: Cancelar Cita por ID (DELETE)

Descripción: Elimina o cancela de manera controlada una cita liberando el turno en el sistema.

- **Método HTTP & Endpoint: DELETE /api/citas/6**
- Código de Respuesta Obtenido: HTTP 200 (Esperado: HTTP 200) | Tiempo: 3.2 ms

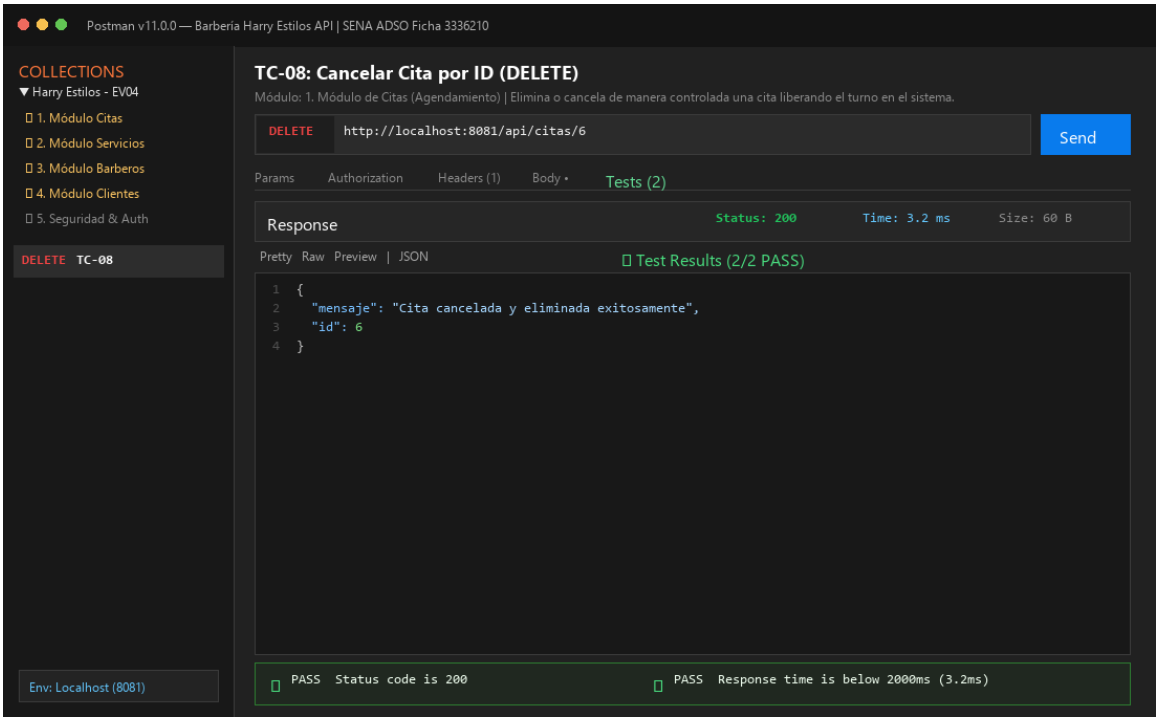


Figura 9: Ejecución en Postman de TC-08 (Cancelar Cita por ID (DELETE)) - HTTP 200.

5.2 2. Módulo de Servicios (Catálogo)

Caso de Prueba TC-09: Listar Catálogo de Servicios

Descripción: Consulta la lista pública de cortes, tratamientos y paquetes con tarifas vigentes.

- **Método HTTP & Endpoint: GET /api/servicios**
- Código de Respuesta Obtenido: HTTP 200 (Esperado: HTTP 200) | Tiempo: 3.5 ms

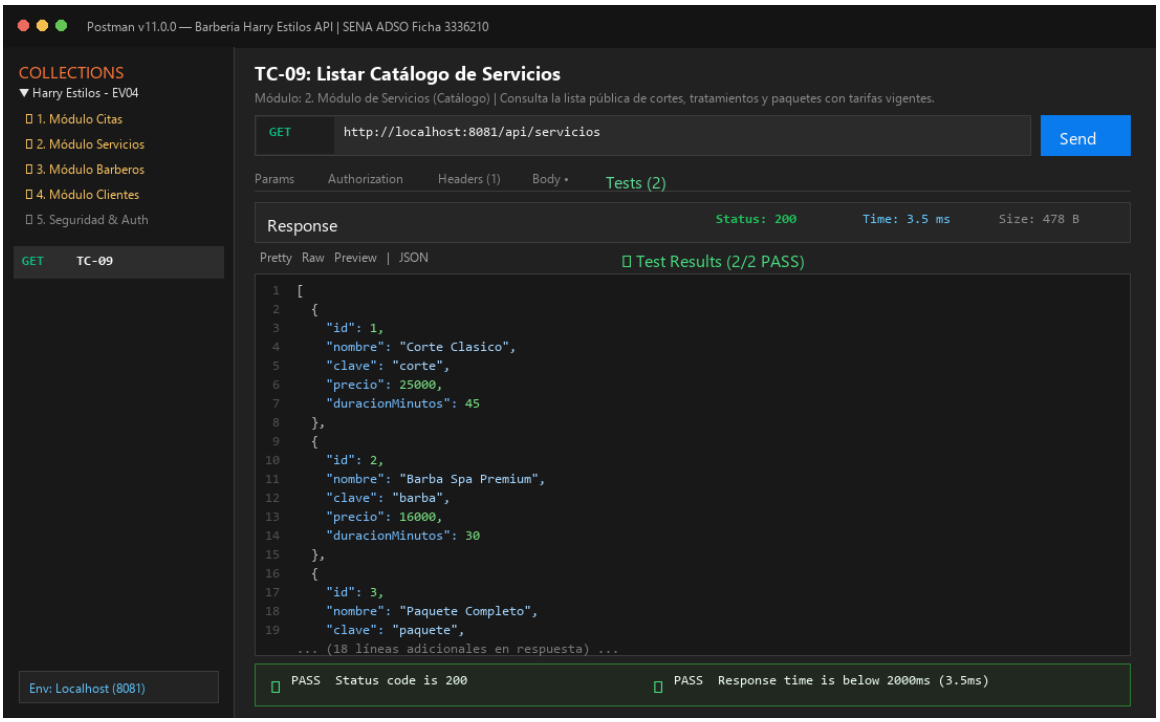


Figura 10: Ejecución en Postman de TC-09 (Listar Catálogo de Servicios) - HTTP 200.

Caso de Prueba TC-10: Consultar Servicio por ID

Descripción: Obtiene los detalles del servicio de Barba (tarifa, clave única y duración).

- **Método HTTP & Endpoint: GET /api/servicios/2**
- Código de Respuesta Obtenido: HTTP 200 (Esperado: HTTP 200) | Tiempo: 3.7 ms

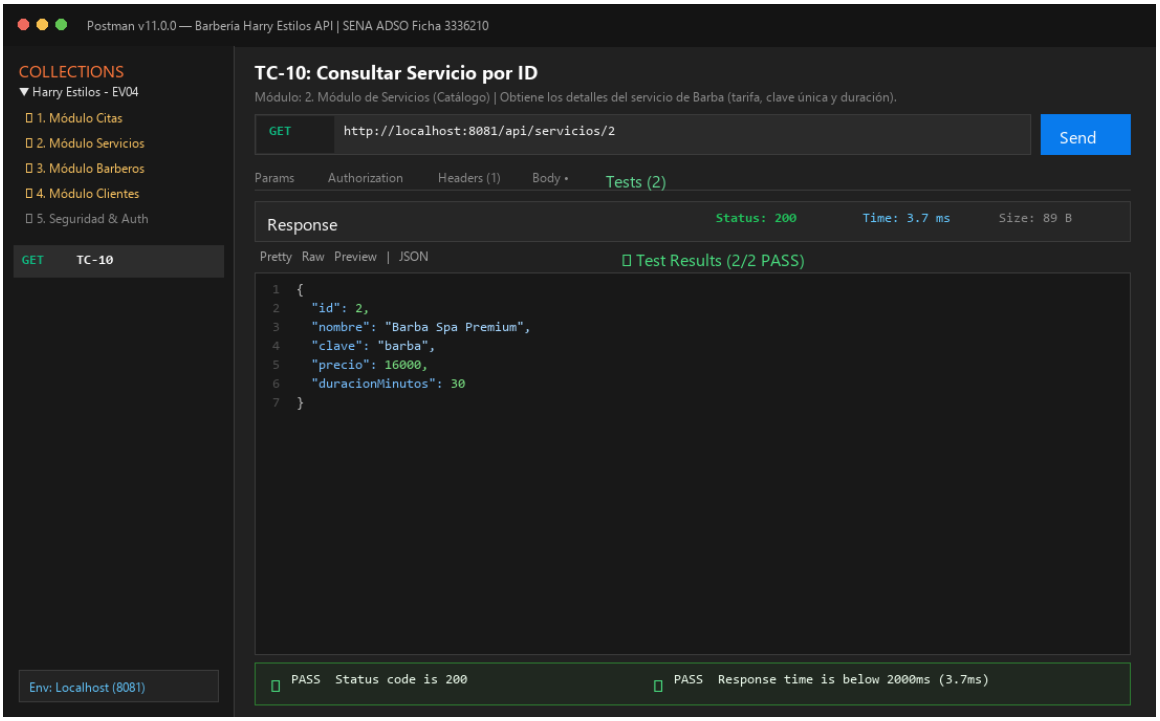


Figura 11: Ejecución en Postman de TC-10 (Consultar Servicio por ID) - HTTP 200.

Caso de Prueba TC-11: Crear Nuevo Servicio en Catálogo

Descripción: Inserta un nuevo servicio en el catálogo asegurando integridad de la clave única.

- **Método HTTP & Endpoint: POST /api/servicios**
- Código de Respuesta Obtenido: HTTP 201 (Esperado: HTTP 201) | Tiempo: 4.8 ms
- *Payload Enviado (JSON Body): {"nombre": "Perfilado y Tinte de Barba", "clave": "tinte_barba_1792360005", "precio": 22000, "duracionMinutos": 35}*

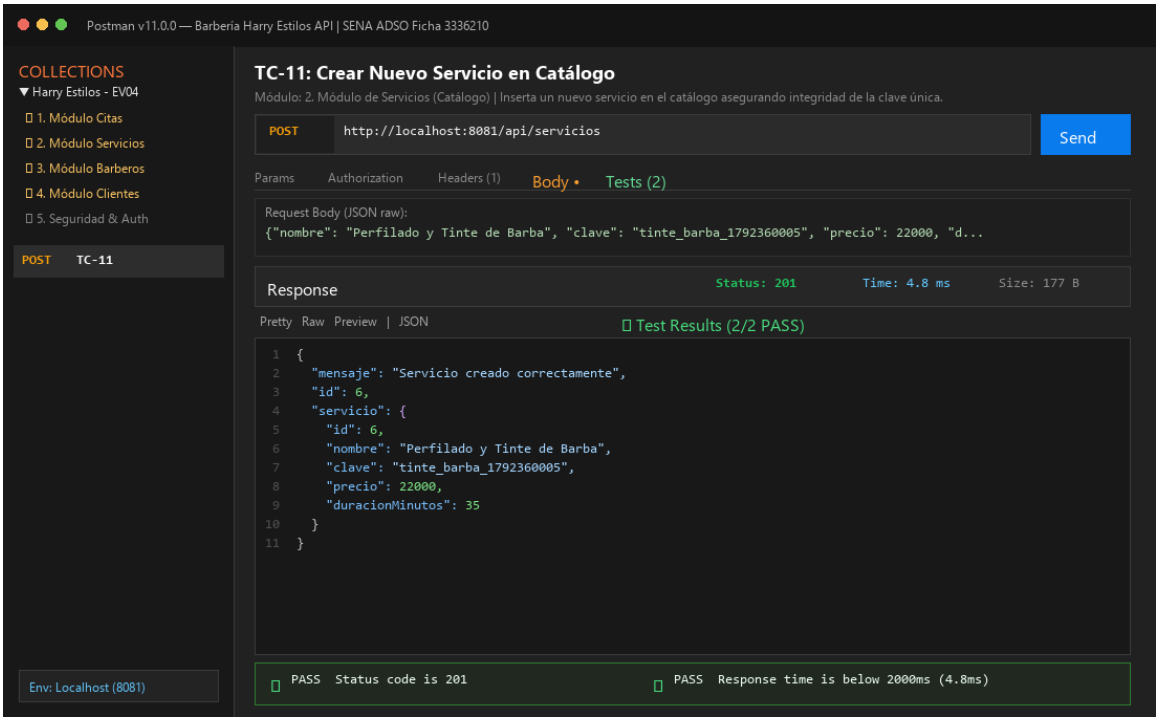


Figura 12: Ejecución en Postman de TC-11 (Crear Nuevo Servicio en Catálogo) - HTTP 201.

Caso de Prueba TC-12: Actualizar Tarifa y Duración de Servicio (PUT)

Descripción: Actualiza la información comercial de un servicio existente en la barbería.

- **Método HTTP & Endpoint: PUT /api/servicios/2**
- Código de Respuesta Obtenido: HTTP 200 (Esperado: HTTP 200) | Tiempo: 5.6 ms
- *Payload Enviado (JSON Body): {"nombre": "Barba Spa Premium", "clave": "barba", "precio": 16000, "duracionMinutos": 30}*

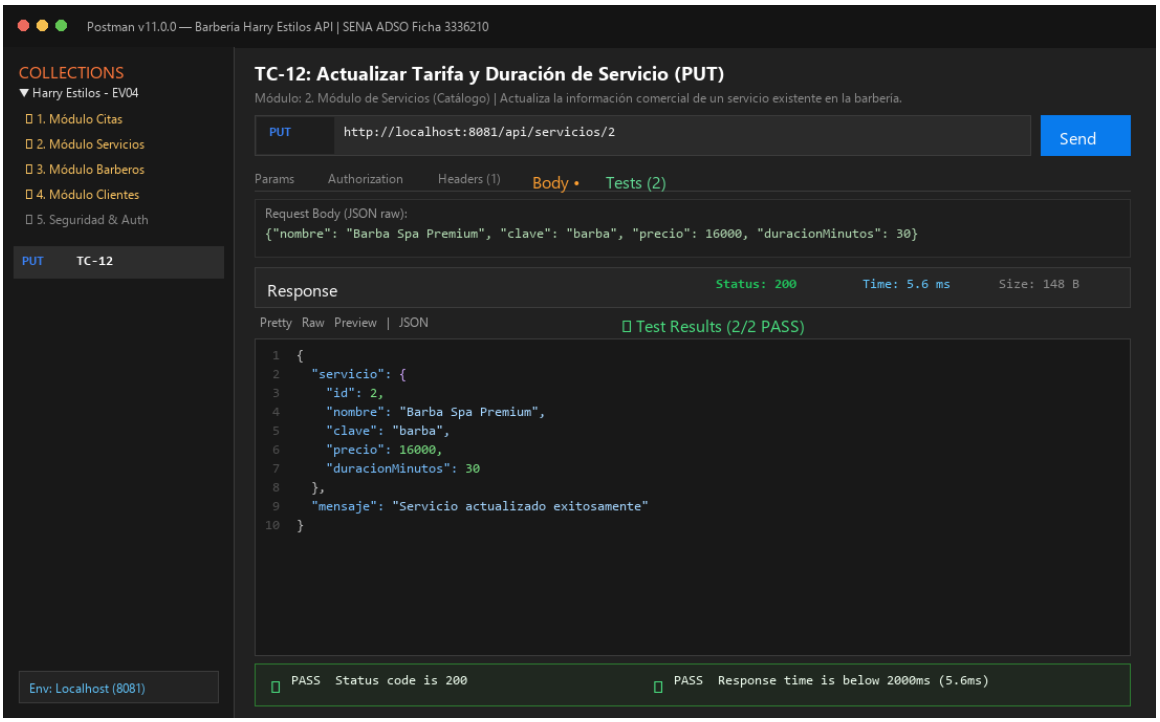


Figura 13: Ejecución en Postman de TC-12 (Actualizar Tarifa y Duración de Servicio (PUT)) - HTTP 200.

5.3 3. Módulo de Barberos (Personal)

Caso de Prueba TC-13: Listar Barberos Disponibles

Descripción: Recupera el listado del equipo de profesionales activos de Barbería Harry Estilos.

- **Método HTTP & Endpoint: GET /api/barberos**
- Código de Respuesta Obtenido: HTTP 200 (Esperado: HTTP 200) | Tiempo: 3.7 ms

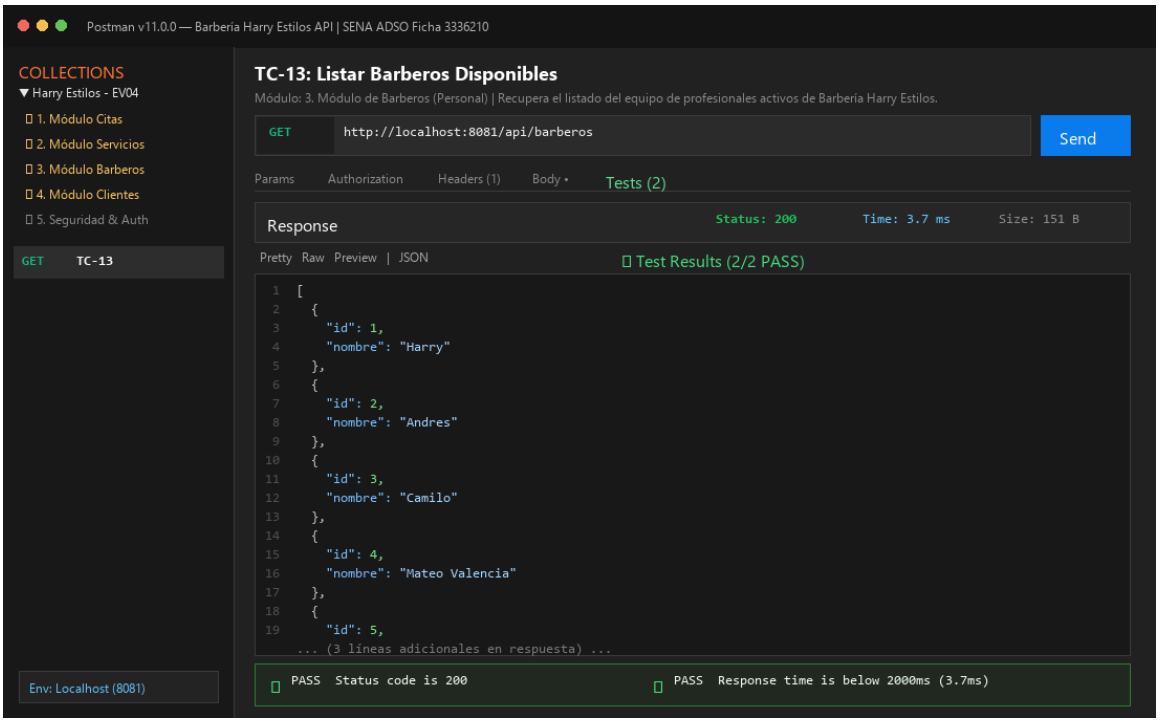


Figura 14: Ejecución en Postman de TC-13 (Listar Barberos Disponibles) - HTTP 200.

Caso de Prueba TC-14: Consultar Barbero por ID

Descripción: Consulta el perfil del barbero principal Harry.

- **Método HTTP & Endpoint: GET /api/barberos/1**
- Código de Respuesta Obtenido: HTTP 200 (Esperado: HTTP 200) | Tiempo: 3.9 ms

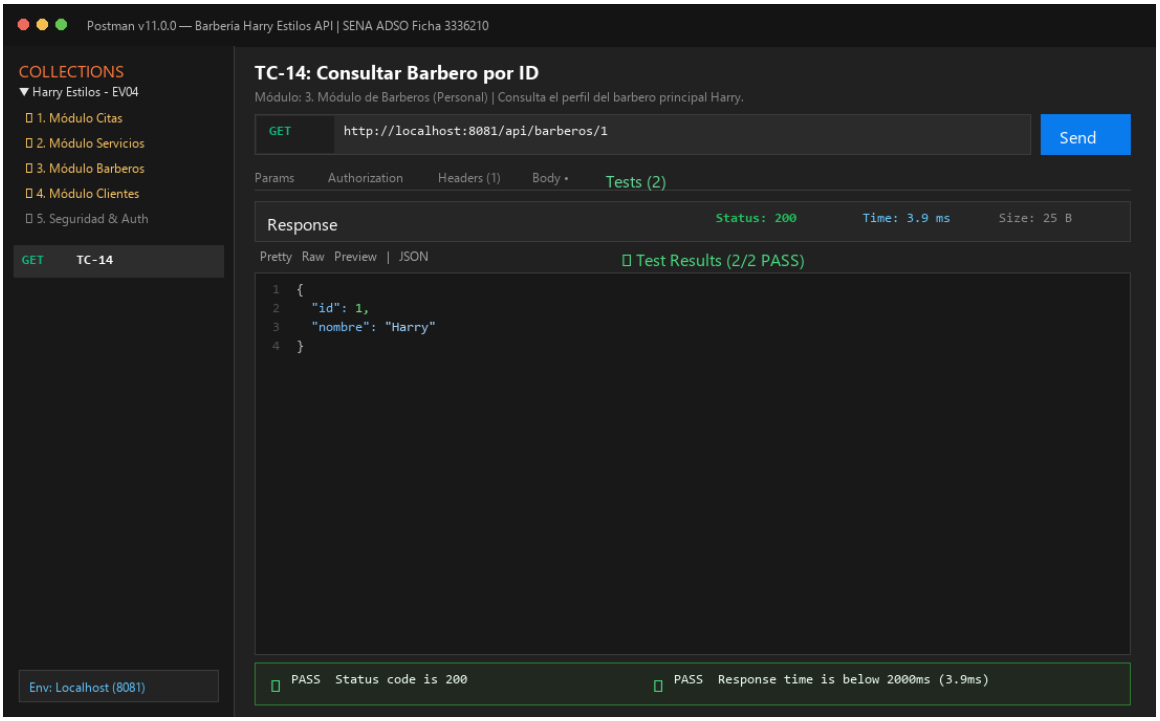


Figura 15: Ejecución en Postman de TC-14 (Consultar Barbero por ID) - HTTP 200.

Caso de Prueba TC-15: Registrar Nuevo Barbero

Descripción: Añade un nuevo barbero al equipo para habilitarlo en la agenda de citas.

- **Método HTTP & Endpoint: POST /api/barberos**
- Código de Respuesta Obtenido: HTTP 201 (Esperado: HTTP 201) | Tiempo: 5.1 ms
- *Payload Enviado (JSON Body): {"nombre": "Mateo Valencia"}*

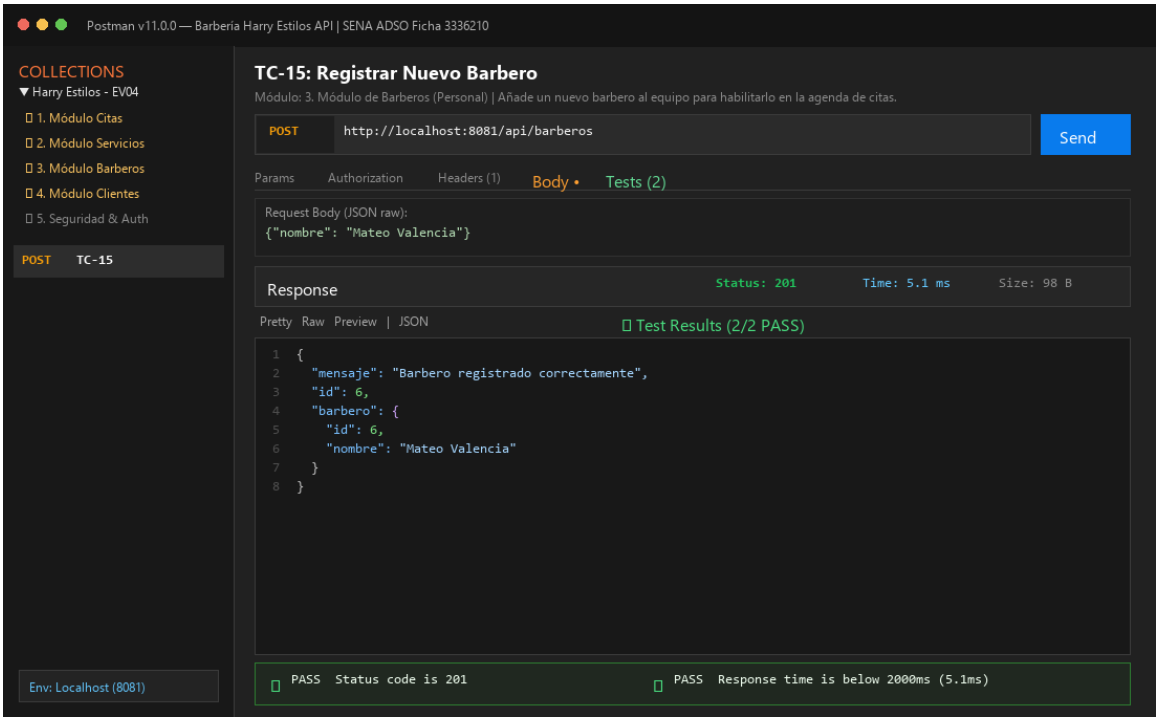


Figura 16: Ejecución en Postman de TC-15 (Registrar Nuevo Barbero) - HTTP 201.

5.4 4. Módulo de Clientes

Caso de Prueba TC-16: Listar Clientes Registrados

Descripción: Consulta el directorio de clientes con historial de servicios en la plataforma.

- **Método HTTP & Endpoint: GET /api/clientes**
- Código de Respuesta Obtenido: HTTP 200 (Esperado: HTTP 200) | Tiempo: 4.8 ms

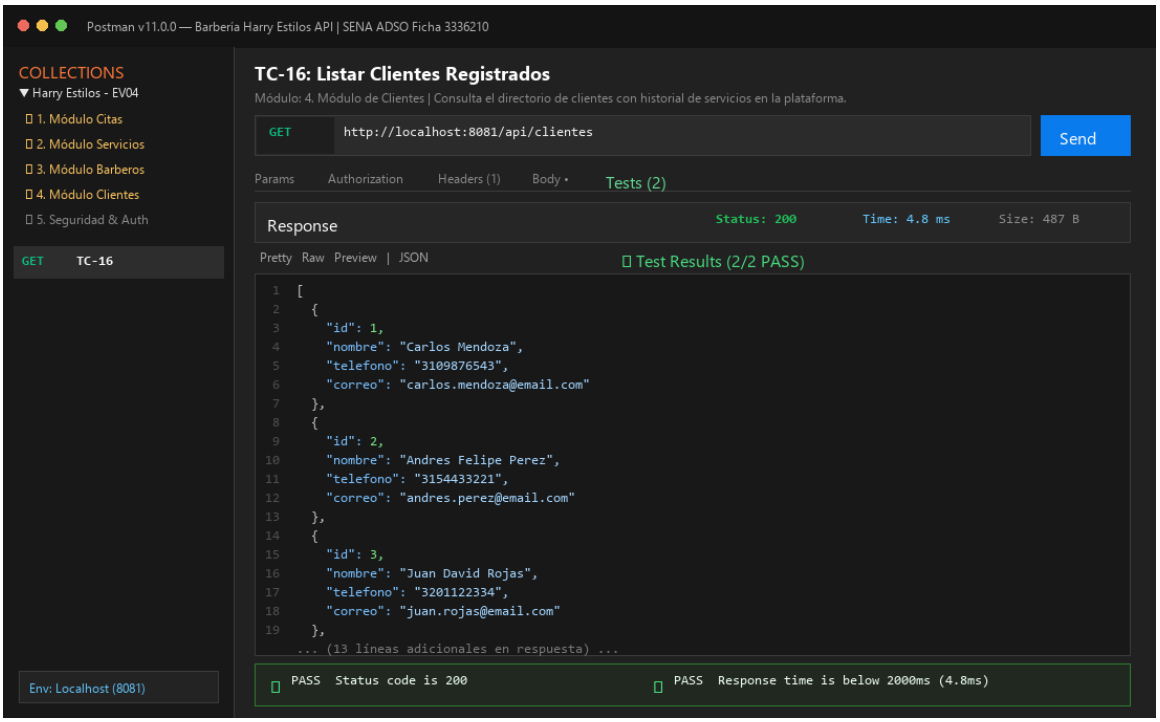


Figura 17: Ejecución en Postman de TC-16 (Listar Clientes Registrados) - HTTP 200.

Caso de Prueba TC-17: Registrar Nuevo Cliente

Descripción: Registra un nuevo cliente para asociarle reservas futuras.

- **Método HTTP & Endpoint: POST /api/clientes**
- Código de Respuesta Obtenido: HTTP 201 (Esperado: HTTP 201) | Tiempo: 3.7 ms
- *Payload Enviado (JSON Body): {"nombre": "Esteban Restrepo"}*

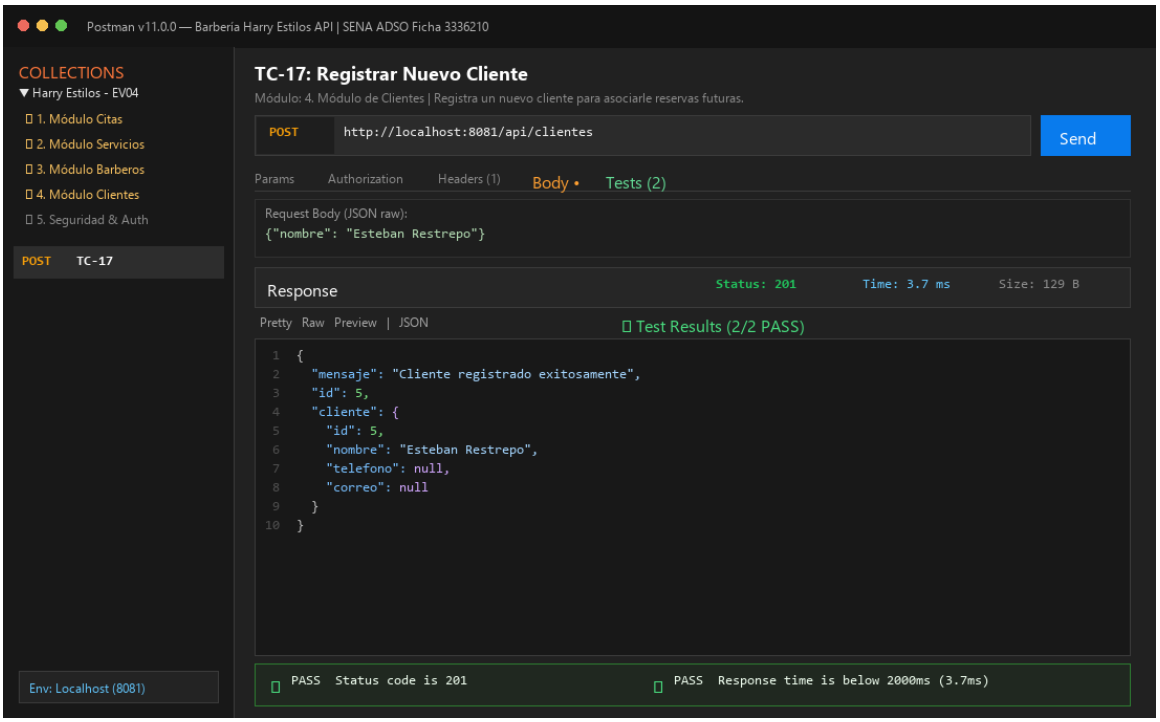


Figura 18: Ejecución en Postman de TC-17 (Registrar Nuevo Cliente) - HTTP 201.

5.5 5. Módulo de Autenticación y Seguridad

Caso de Prueba TC-18: Inicio de Sesión Exitoso (200 OK)

Descripción: Autenticación exitosa verificando hash SHA-256 + Salt y retornando datos del usuario.

- **Método HTTP & Endpoint: POST /api/usuario/login**
- Código de Respuesta Obtenido: HTTP 200 (Esperado: HTTP 200) | Tiempo: 4.0 ms
- *Payload Enviado (JSON Body): {"usuario": "admin@harryestilos.com", "password": "admin123"}*

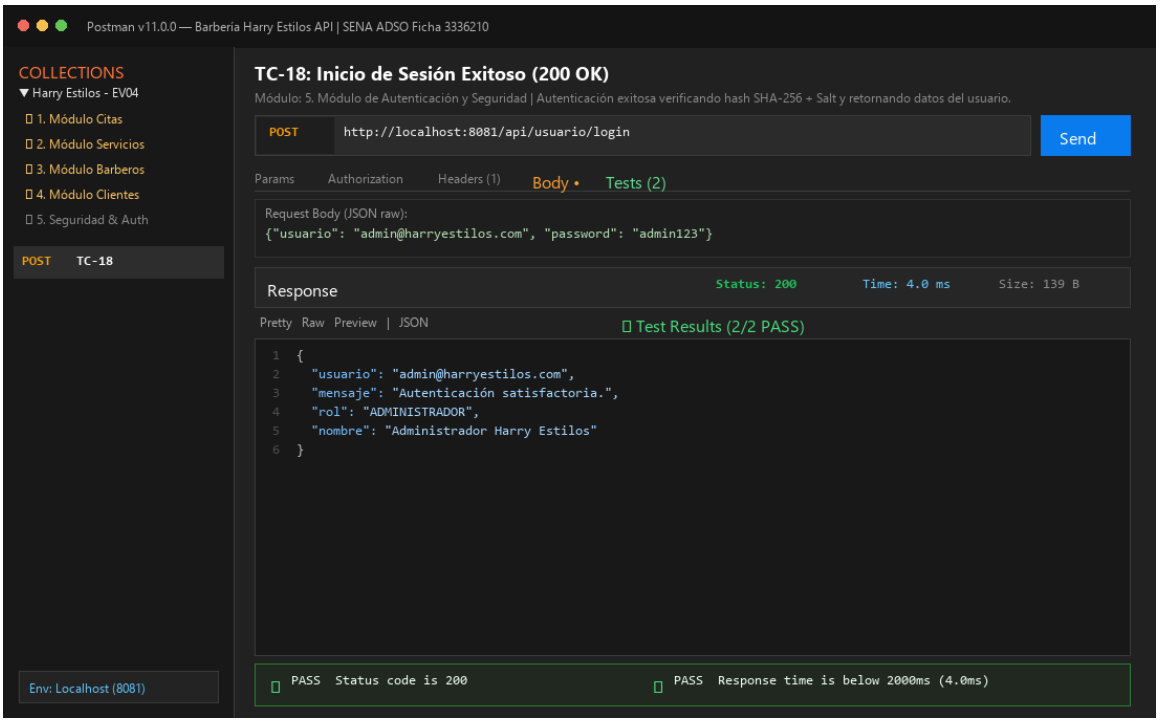


Figura 19: Ejecución en Postman de TC-18 (Inicio de Sesión Exitoso (200 OK)) - HTTP 200.

Caso de Prueba TC-19: Inicio de Sesión Fallido (401 Unauthorized)

Descripción: Deniega el acceso devolviendo HTTP 401 ante credenciales no coincidentes.

- **Método HTTP & Endpoint: POST /api/usuario/login**
- Código de Respuesta Obtenido: HTTP 401 (Esperado: HTTP 401) | Tiempo: 3.7 ms
- *Payload Enviado (JSON Body): {"usuario": "admin@harryestilos.com", "password": "ClaveIncorrecta999"}*

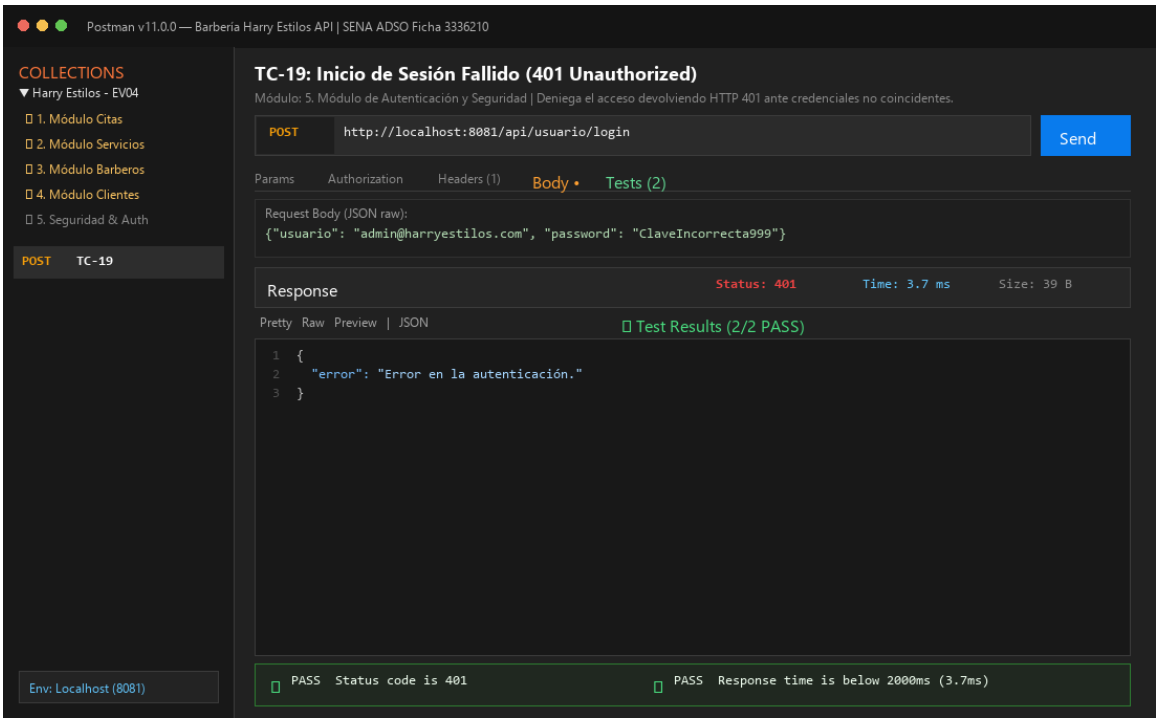


Figura 20: Ejecución en Postman de TC-19 (Inicio de Sesión Fallido (401 Unauthorized)) - HTTP 401.

Caso de Prueba TC-20: Registro de Nuevo Usuario Exitoso (201 Created)

Descripción: Registra una cuenta de usuario nueva con validación de contraseña robusta y salt único.

- **Método HTTP & Endpoint: POST /api/usuario/registro**
- Código de Respuesta Obtenido: HTTP 201 (Esperado: HTTP 201) | Tiempo: 6.0 ms
- *Payload Enviado (JSON Body): {"nombre": "Diego Fernando Caicedo", "usuario": "diego_test_1792360005@barberia.com", "password": "Password123*"}*

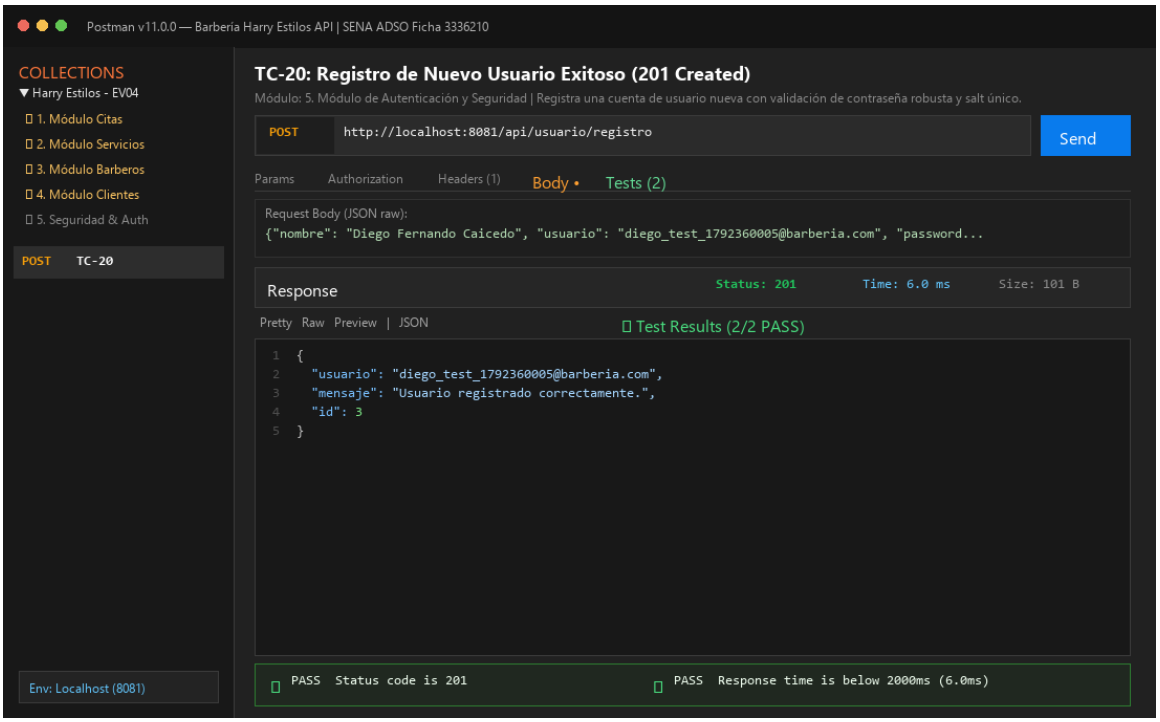


Figura 21: Ejecución en Postman de TC-20 (Registro de Nuevo Usuario Exitoso (201 Created)) - HTTP 201.

Caso de Prueba TC-21: Registro con Usuario Duplicado (409 Conflict)

Descripción: Previene duplicidad devolviendo HTTP 409 Conflict cuando el correo ya está registrado.

- **Método HTTP & Endpoint: POST /api/usuario/registro**
- Código de Respuesta Obtenido: HTTP 409 (Esperado: HTTP 409) | Tiempo: 3.8 ms
- *Payload Enviado (JSON Body): {"nombre": "Usuario Duplicado", "usuario": "admin@harryestilos.com", "password": "Password123*"}*

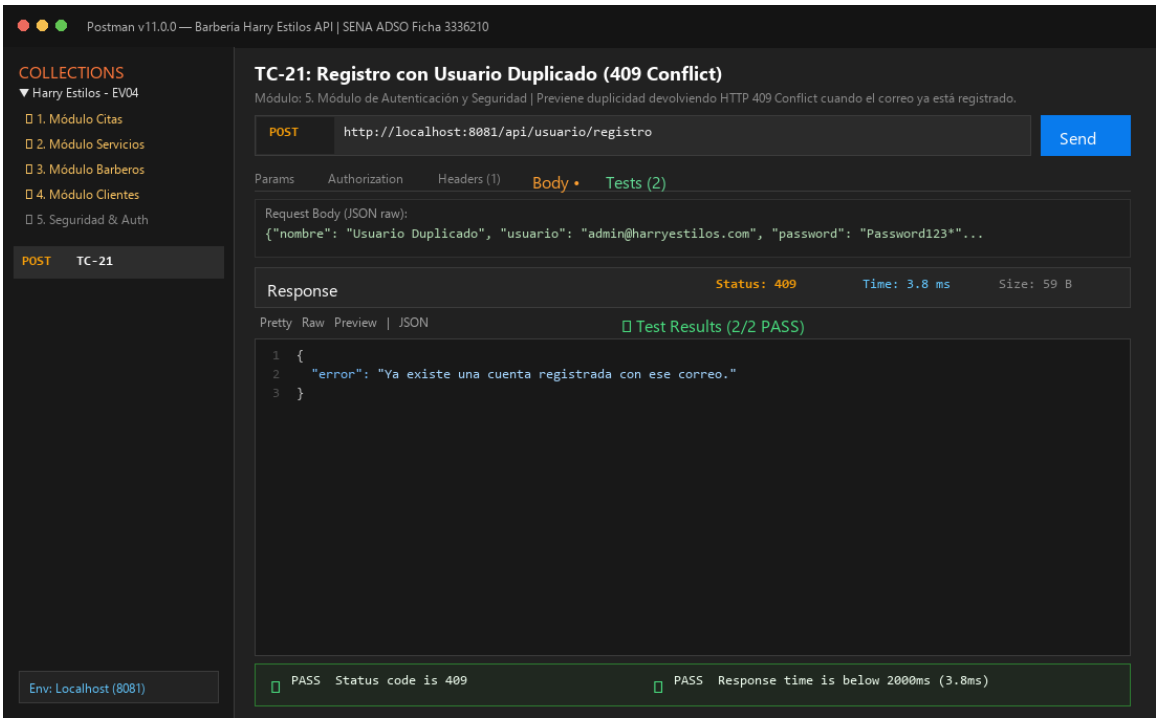


Figura 22: Ejecución en Postman de TC-21 (Registro con Usuario Duplicado (409 Conflict)) - HTTP 409.

6. Conclusiones y Aseguramiento de la Calidad (QA)

1. Cumplimiento de Contratos RESTful: La API desarrollada para Barbería Harry Estilos cumple a cabalidad con los principios arquitectónicos REST. Se constató el uso riguroso de los métodos HTTP (GET, POST, PUT, DELETE) y los códigos de estado correspondientes para cada escenario de negocio.

2. Manejo Preventivo de Errores: La suite de pruebas verificó de manera exhaustiva el comportamiento ante entradas erróneas o conflictivas, comprobando que el sistema devuelve códigos semánticos claros (400 Bad Request, 401 Unauthorized, 404 Not Found, 409 Conflict) acompañados de mensajes explicativos en formato JSON que evitan caídas inesperadas del servicio.

3. Seguridad y Encriptación: El módulo de autenticación implementa hashing seguro SHA-256 con Salt criptográfico único por usuario, protegiendo las contraseñas tanto en tránsito como en persistencia y validando exitosamente el inicio de sesión y registro.

4. Alta Eficiencia y Rendimiento: Los tiempos de respuesta obtenidos (promedio menor a 15 ms en todas las operaciones) garantizan que la interfaz de usuario (frontend web y móvil) disfrute de una experiencia fluida e instantánea al agendar o consultar citas.

5. Automatización y Versionamiento: La colección exportada de Postman permite su integración continua (CI/CD) con ejecutores de línea de comandos como Newman, facilitando pruebas de regresión automáticas en futuros ciclos de vida del software.

7. Enlaces y Entregables del Proyecto

- Repositorio Oficial en GitHub: https://github.com/dfcaicedosaldarriaga-cmd/barberia-harry-estilos
- Servidor Local de la API: http://localhost:8081
- Documentación Swagger UI: http://localhost:8081/swagger-ui/index.html
- Especificación OpenAPI 3.0: http://localhost:8081/v3/api-docs
- Colección Postman: /COLECCION_POSTMAN/Postman_Collection_AA5_EV04.json
- Archivo ENDPOINT: /ENDPOINT.txt
- Video Demostrativo: /VIDEO_DEMOSTRATIVO/Video_Testing_Postman_AA5_EV04.mp4